

Competitive Programming

Section 49: Convex Hull Trick And Li Chao Tree

Competitive Programming Course

2026-07-10

Table of contents

1	Lines as DP states	3
1.1	Turning a DP into lines	3
1.2	Convex hull trick assumptions	3
1.3	Li Chao tree invariant	3
1.4	Why local exercises may use brute force	4
1.5	Overflow checklist	4
1.6	Line intersection intuition	4
1.7	DP expansion checklist	4
1.8	Minimal line object	5
1.9	Testing line containers	5
1.10	When CHT is not the right tool	5
2	Exercises	6
3	Line-container invariant	7

1 Lines as DP states

Many transitions have the form $dp[j] + m_j * x_i + b_j$. Treat each previous state as a line and each current state as a query point. This section uses small direct versions to expose the algebra, then frames the Li Chao/CHT invariant: keep the best line for intervals of x -values.

1.1 Turning a DP into lines

Consider:

$$dp[i] = \min_j dp[j] + (x_i - x_j)^2 + C$$

Expand the square:

$$dp[i] = x_i^2 + C + \min_j (dp[j] + x_j^2 - 2x_jx_i)$$

For a fixed j , this is a line queried at $x = x_i$:

$$\begin{aligned} m &= -2x_j \\ b &= dp[j] + x_j^2 \\ \text{line}(x_i) &= m * x_i + b \end{aligned}$$

The term $x_i^2 + C$ is the same for every candidate j , so add it after the line query.

1.2 Convex hull trick assumptions

The simplest deque CHT requires monotone slopes and monotone query x values. That is common in AtCoder DP Z style problems when x_i is sorted. If either order is arbitrary, use a Li Chao tree or a balanced hull.

1.3 Li Chao tree invariant

A Li Chao segment tree stores one line at each node interval $[l, r]$. The stored line is the best known line at the midpoint among lines assigned to that node's interval. When inserting a new line:

1. Compare old and new at l , mid , and r .
2. Keep the line that is better at mid in the current node.
3. Recurse with the worse-at- mid line into the side where it might still win.

For minimum queries, “better” means smaller value.

```
interval [l.....mid.....r]
if new wins at mid, swap(new, stored)
the losing line can only beat stored on one side
```

A query at x walks from root to leaf and takes the minimum of all stored lines encountered.

1.4 Why local exercises may use brute force

The local judge keeps limits small enough for transparent reference solutions, so the full code is easy to inspect. The lesson still teaches the red-level reduction because the external practice tasks require the optimized structure. When upsolving those, replace the linear scan over lines with Li Chao or CHT without changing the mathematical line model.

1.5 Overflow checklist

- Use 64-bit for slopes, intercepts, and answers.
- If coordinates and coefficients can reach $1e9$, products reach $1e18$.
- In C++, use `__int128` inside evaluation if constraints approach $1e18$.

1.6 Line intersection intuition

For two lines:

```
y = m1*x + b1
y = m2*x + b2
```

one is better on one side of their intersection, and the other is better on the other side. Li Chao trees avoid explicitly storing all intersections. At each interval midpoint, they keep the line that wins there and pass the other line only to the side where it may still win.

That is why Li Chao insertion is logarithmic over a fixed coordinate range: each recursive step discards one half of the interval for the losing line.

1.7 DP expansion checklist

When you suspect CHT:

1. Write the transition.
2. Expand products and squares.
3. Separate terms depending only on i .
4. Express the remaining j -dependent part as $m_j * x_i + b_j$.
5. Decide whether slopes/queries are monotone; if not, use Li Chao.

1.8 Minimal line object

C++:

```
struct Line {  
    long long m, b;  
    long long get(long long x) const {  
        return m * x + b;  
    }  
};
```

In a brute-force local solution, a query scans all lines. In a Li Chao solution, the query scans the $O(\log C)$ nodes on the root-to-leaf path for coordinate range size C . The only difference is the data structure used to find the best line; the line model is identical.

1.9 Testing line containers

Always compare against brute force on random small data:

```
insert random lines  
query random x values  
answer = min(line(x) for every inserted line)
```

Include negative slopes, equal slopes, negative intercepts, and repeated query coordinates. Equal slopes are a common source of bugs: for minimum queries, keep only the smaller intercept if two slopes are equal.

1.10 When CHT is not the right tool

Do not force CHT if the transition cannot be written as:

```
 $m_j * x_i + b_j + \text{term\_only\_depending\_on\_i}$ 
```

If the candidate has $x_i * x_j$ plus other mixed terms, try algebra first. If the coefficient of x_i still depends on more than one changing value, the state may need a different optimization or no known optimization at all.

The red-coder skill is recognizing both when the line model exists and when it does not.

2 Exercises

3 Line-container invariant

Each line represents one previous DP state. A Li Chao node stores a line that is best at its midpoint; any displaced line is recursively retained only where it can still win. Therefore insertion preserves the minimum envelope and query follows one root-to-leaf path. Use `long long` or wider products, test equal slopes, negative coordinates, and queries outside no declared domain.

- A. Dynamic Line Minimum: Add lines $y=mx+b$ and answer minimum value at query points.
- B. Quadratic CHT DP: Compute $dp[i]=\min_j dp[j]+(x_i-x_j)^2+c$ for increasing x .
- C. Dynamic Line Maximum: Add lines and answer maximum value at query points.